

PilotFish FTP / SFTP Listener – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **remote folder watcher over FTP**. It connects to an FTP, FTPS, or SFTP server on a schedule, lists files in a configured directory, downloads matches, and starts a PilotFish transaction for each one. After pickup it can delete, move, or leave the remote file.

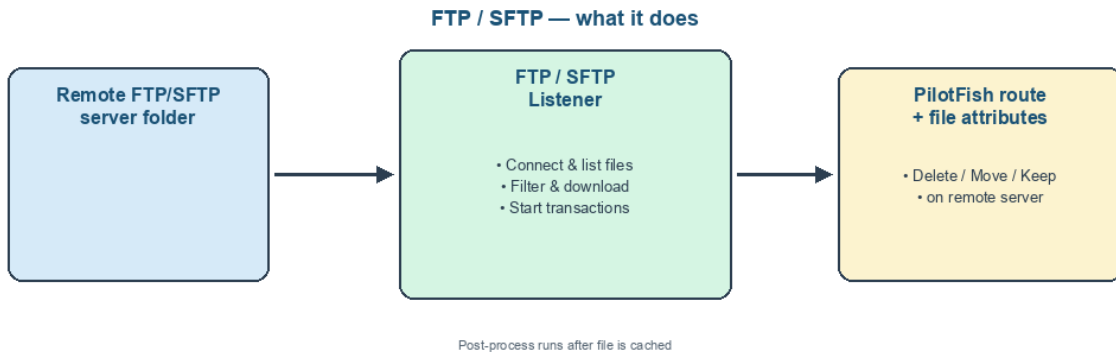


Figure 1: At a glance (plain English)

- **Polls** a remote directory on a timer (like the local Directory listener).
- **Supports** regular FTP, explicit/implicit FTPS, and two SFTP stacks.
- **Filters** by name, extension, permissions, and write-in-progress heuristics.
- **Cleans up** remote files after the listener caches them (not after route success).

Purpose

Deep dive into the **FTP / SFTP** listener in `eip.war.hs.26R1.11`: protocol selection, polling, file acceptance, caching, post-process, and configuration.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-file-1.0.1.jar</code>
Core JAR	<code>xcs-core-1.2.6.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.file.ftp.FTPList</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../FT</code>
Listener type string	<code>FTP / SFTP</code>
Description	Monitors a specified directory on a remote FTP/SFTP/FTPS server for new files.
Help ID	<code>console.listener.ftp</code>

Related classes:

Class	Role
<code>FTPListener</code>	Polling orchestration, handler factory, timeout wrapper
<code>FtpListenerHandler / SFTPListenerHandler</code>	Protocol-specific list/download/post-process
<code>FtpListenerCommons</code>	Accept filters, attributes, cache/reconnect
<code>AutoDeleteFTPFile</code>	Remote delete with retry
<code>FtpCommons</code>	Credentials, FTPS certificates, SSL options

1. High-level behavior

1. **Engage:** single-thread scheduler runs `doListening()` every **Polling Interval** (seconds), initial delay 1s.
2. **Each poll:** work runs on a separate **FTP Listener Runner Thread** with optional **Connection timeout** cap.
3. **Handler selection** by FTP Type: regular FTP, explicit/implicit FTPS, SFTP (SSHTools), SFTP (JSCH).
4. **Directory mode:** increment polling cycle ID, reset **Max Transactions Per Run** quota, list/recurse remote folder.
5. **Full path mode:** pick up one file (optional MVS data set semantics).
6. **Per file:** download to File or Memory cache, dispatch, post-process (Delete/Move/Keep) after cache fill.
7. **Disengage:** shutdown scheduler. **Keep connection** false closes handler after each poll.

```

doListening()
    FutureTask on runner thread
    handler = getListenerHandler(keepConnection)
    connect if needed
    [UseFullFilePath?] pickup single file / MVS DSN
    else pickupFtpFilesInDirectory (quota + recursion)
    sendFTPTransactions(cache, attrs) -> dispatchListeners

finally: if !keepConnection -> handler.closeConnections()

```

2. Configuration reference

2.1 Basic

Label	Tag	Default	Notes
FTP Type	FTPListenerType	Regular FTP	5 choices (see below)
Host	Host	""	Required
Port	Port	21	21 FTP/FTPS, 22 SFTP typical
Polling Interval	PollingInterval	30	Seconds
Polling directory	PollingDirectory	""	. or / for cwd/root
File/folder name restriction	FileNameRestriction	""	Substring or full regex
File extension restriction	FileExtensionRestriction	""	Comma-separated list
Specify full file path	UseFullFilePath	Disabled	MVS/single-file mode
Path to file/folder	FullPathToFile	""	Required when full path enabled
File is MVS data set	FilenameIsMvsDsrf	false	Full path + MVS pickup

FTP Type choices: Regular FTP; FTP over SSL (Explicit); FTP over SSL (Implicit); Encrypted FTP (SSH/SFTP); Encrypted FTP (JSCH SSH/SFTP).

2.2 Advanced

Label	Tag	Default	Nuances
Max Transactions Per Run	SerializedTransactionsTag	250	Range 1–255; decremented per file
Connection timeout	Timeout	300	Seconds for entire poll FutureTask; 0 = unlimited
Socket Timeout	Sftp_SocketTimeout	0	SFTP connect wait; 0 = library default 30s

Label	Tag	Default	Nuances
Retry Connection	RetryConnection	false	Second attempt after failure
Retry Delay	RetryDelay	1	Seconds; -1 = no delay
Keep connection	KeepConnection	false	Reuse handler keyed by connection settings
Passive Mode	UsePassiveMode	true	FTP/FTPS only
SSL Protocol	SSLProtocol	TLS	Explicit/implicit FTPS
Data Channel Protection Level	Protection	Private (P)	FTPS
Protection Buffer Size (PBSZ)	ProtectionBufferSize	0	FTPS
File Type	FileType	Binary	Binary vs Text (ASCII)
Stop if directory not found	StopWhenPollingFailure	Failure	Error when poll/postprocess dir missing
Include sub-folders	SubFolderIterationTag	Hidden	Hidden when UseChangeDirectory = true
Use ChangeDirectory Command	UseChangeDirectoryCommand	false	Explicit CWD vs path-in-name
Reconnect After Bytes	ReconnectAfterBytes	102400	SFTP only; force reconnect after N bytes
Cache Type	CacheType	File Cache	File Cache vs Memory Cache

2.3 Post-process tab

Label	Tag	Default
Postprocess operation	PostProcessOperation	Delete
Target directory	TargetDirectory	"" (local filesystem for Move)

Move shows **Target directory** (local path, writeability checked). Remote delete/move uses **AutoDeleteFTPFile** with up to 10 retries.

2.4 Accept Tests tab

Label	Tag	Default	Behavior
Check FTP Permissions	CheckFTPPermissions	true	Requires WRITE on remote file for delete/move
Check Previously Processed	CheckPreviouslyProcessed	true	Skips files in failedDeleteFiles set
Check Write In Progress	CheckWriteInProgress	true	Skips if mtime within 10 seconds

2.5 Credentials & certificates

From `FtpCommons.getCredentialsDescriptor` and `getFtpsCertConfigDescriptor` (username/password, keys, trust stores — vary by FTP Type).

2.6 Scheduling tab

Inherited scheduler configuration (start/end, exclusions, time zone).

3. File acceptance algorithm (`FtpListenerCommons`)

For each remote file candidate:

1. **Directories** always pass **accept** (for recursion).
2. **Name restriction:** base name must contain restriction substring **or** match as full regex.
3. **Extension restriction:** extension must be in comma list (via `Util.getExtension`).
4. **gpmAccept:**
 - Reject if Check FTP Permissions and no WRITE permission.
 - Reject if Check Write In Process and modified < 10s ago.
 - Reject if Check Previously Processed and fingerprint in `failedDeleteFiles`.

Quota: `serializedTxLeft` initialized from Max Transactions Per Run each directory poll; decremented per pickup.

4. Transaction attributes

Set in `FtpListenerCommons.createTransactionAttributes`:

Attribute	Type	Meaning
<code>com.pilotfish.SourceFile</code>	Object (protocol file handle)	Remote file reference
<code>com.pilotfish.FileName</code>	String	Base name without extension
<code>com.pilotfish.FileExtension</code>	String	Extension
<code>com.pilotfish.FileSize</code>	Long	Remote size
<code>com.pilotfish.FTPHost</code>	String	Configured Host
<code>com.pilotfish.PollingCycleID</code>	Integer	Incremented each directory poll
<code>com.pilotfish.PollingCycleSize</code>	Integer	Batch size for cycle
<code>com.pilotfish.RelativeSourceFolderPath</code>	String	Relative remote folder (not MVS mode)

5. Connection & timeout model

- **Keep connection:** handler stored in `liveHandlers` map keyed by `factory.generateKeepConnectionKey()`

- **Poll timeout:** if `Connection timeout > 0`, `FutureTask.get(timeout, SECONDS)`; on time-out/interrupt, future cancelled, connection not kept, error raised.
- **SFTP reconnect:** after Reconnect After Bytes received, handler closes and reconnects mid-transfer.

6. Worked example (quota + subfolders)

Max Transactions Per Run = 2, Include sub-folders = true, polling dir `/inbound`:

```
/inbound/a.dat
/inbound/b.dat
/inbound/sub/c.dat
```

Cycle 1: takes `a.dat`, `b.dat` (order depends on listing). Quota 0 — `sub/c.dat` waits. Cycle 2 picks up `c.dat` if still present and matching.

7. Known quirks

1. **Post-process timing:** delete/move after download to cache, not after route completes.
2. **Move target is local filesystem**, not remote — remote Move may mean delete after local copy depending on handler.
3. **Windows FTP:** disable Check FTP Permissions if WRITE bits are unreliable.
4. **failedDeleteFiles** uses hash + mtime fingerprint — prevents reprocessing after failed delete.
5. **Connection timeout** cancels in-flight poll thread — partial pickups possible.
6. **Use ChangeDirectory** disables sub-folder iteration in UI dependencies.
7. **MVS mode** skips `RelativeSourceFolderPath` attribute.
8. **Immediate save** enabled on descriptor.

8. Operational recommendations

Goal	Settings
Large files	File Cache; SFTP Reconnect After Bytes
Many small files	Memory Cache; higher Max Transactions Per Run
Unstable network	Retry Connection; finite Connection timeout
Exactly-once handoff	Move to archive path; monitor failedDeleteFiles
FTPS interoperability	Try TLS + Passive Mode; tune Protection level
Windows FTP server	Check FTP Permissions = false

9. Inherited AbstractListener options

Same base set as other listeners: Initialize on trigger only (`IS_TRIGGERABLE_LISTENER`), CLI (`CLIAllowed`), Restart on error, FIFO queue, throttling (`None/Timed/Concurrent`), inactivity monitor, scheduling tabs. See Directory Listener reference for dependency details.

10. Source index

Topic	Path
FTPLListener	<code>com/pilotfish/eip/modules/file/ftp/FTPLListener.java</code>
FtpListenerCommons	<code>com/pilotfish/eip/modules/file/ftp/FtpListenerCommons.java</code>
AutoDeleteFTPFile	<code>com/pilotfish/eip/modules/file/ftp/AutoDeleteFTPFile.java</code>
FtpCommons	<code>com/pilotfish/eip/modules/file/ftp/FtpCommons.java</code>

Deep-dive documentation from WAR decompile — 26R1.11 — August 2026.